

Chapter 1

Reinforcement Learning and Decision-Making Problem

Learning often takes place through interaction with the world around us. From an early age, people learn by observing how their actions affect their surroundings. For example, when a child learns to ride a bicycle, they gradually discover how changes in balance, speed, and direction influence their movement. Through repeated attempts and feedback from the environment, the child learns which actions are effective and which are not. This type of interaction provides valuable information about cause and effect, consequences, and the actions required to achieve a desired outcome. This process continues throughout our lives. Whether we are learning a new skill, using unfamiliar technology, or solving a practical problem, we often learn by observing the results of our actions and adjusting our behavior accordingly.

Learning through interaction offers important insight into human learning and artificial intelligence. This course examines it computationally—not by directly explaining how humans or animals learn, but by studying simplified environments and comparing computational methods. This reflects an AI researcher’s or engineer’s goal of building machines that learn to solve useful problems.

This approach, known as reinforcement learning and decision-making, involves an agent interacting with an environment: selecting actions, observing consequences, and receiving rewards or other feedback. Over time, the agent learns actions that improve its expected long-term outcomes, distinguishing reinforcement learning from many other machine-learning approaches.

1. Reinforcement Learning and Decision-Making

Reinforcement Learning (RL) is a branch of artificial intelligence and machine learning in which an agent learns how to make decisions by interacting with an environment. Unlike supervised learning, which learns from labeled examples, reinforcement learning learns from experience. The agent performs an action, observes its consequences, receives a reward, and uses this experience to improve its future decisions. The fundamental objective of reinforcement learning is to learn a policy that selects actions in different situations so that the agent maximizes the cumulative reward it receives over time. For example, a robot learning to navigate a building can receive a positive reward when it reaches its destination and a negative reward when it collides with an obstacle. Through repeated interaction, the robot can learn which actions lead to better outcomes.

A major characteristic of reinforcement learning is its interactive, closed-loop nature. The agent takes an action, the environment changes as a result, and the new situation affects what the agent can do next. Unlike supervised learning, the agent is not directly told which action is correct. Instead, it learns through experience by trying different actions and observing their consequences. An action can influence not only the immediate reward but also future states and rewards. Therefore, RL often requires considering long-term consequences rather than focusing only on immediate results. A reinforcement learning problem generally contains three essential components: sensation, action, and goals. The agent must be able to observe part of its environment, take actions that influence the environment, and have a goal that determines which outcomes are desirable. The environment is usually uncertain, so the agent must make decisions without knowing exactly what will happen.

Reinforcement learning differs from supervised learning. In supervised learning, an external source provides labeled examples showing the correct response for each situation. The system then learns to generalize from these examples to new situations. In reinforcement learning, however, it may be impossible to provide correct examples for every situation. The agent, therefore, needs to learn from its own interaction and experience. RL is also different from unsupervised learning. Unsupervised learning generally attempts to discover patterns or structures in data without labeled answers. Reinforcement learning, in contrast, focuses on maximizing a reward signal. Although

discovering patterns can help an RL system, finding structure alone does not solve the problem of choosing actions that maximize reward. For this reason, reinforcement learning is commonly treated as a distinct machine learning paradigm.

One of the central challenges in RL is the exploration–exploitation dilemma. The agent must exploit actions that it already knows can produce good rewards, but it must also explore unfamiliar actions to discover potentially better ones. Relying entirely on exploitation may prevent the agent from finding better strategies, while relying entirely on exploration may prevent it from gaining useful rewards. Therefore, an effective agent needs to balance both. Another important feature of reinforcement learning is its focus on a complete goal-directed agent interacting with an uncertain environment. RL considers sensing, decision-making, action, learning, and long-term goals as connected parts of a larger problem. Planning can be included, but RL also considers how planning interacts with real-time decisions and how environmental models can be learned and improved. Reinforcement learning has connections with several fields, including statistics, optimization, control theory, psychology, and neuroscience. Some RL techniques help address problems involving very large numbers of possible states and actions. RL has also been influenced by research on how humans and animals learn, while RL models have, in turn, contributed to theories of biological learning and reward systems.

Examples of Reinforcement Learning: Reinforcement learning can be illustrated through many real-world situations:

Chess: A player chooses moves while considering possible future positions and rewards.

Industrial control: A controller can adjust a refinery's operations to improve production, costs, or quality.

Animal learning: A young gazelle learns to stand and run through interaction with its environment.

Mobile robots: A robot may decide whether to explore for objects or return to its charging station based on its battery level and previous experience.

Everyday activities: Even preparing breakfast involves many decisions, actions, observations, and goals.

These examples share several important characteristics. In each case, an agent interacts with an environment, has a goal, makes decisions under uncertainty, and takes actions that can affect future situations. Because actions may have delayed consequences, the agent often needs to consider the future and adjust its behavior in light of new information. The agent does not have to be a whole person or a robot. It can be a smaller decision-making system within a larger organism or machine. Similarly, the environment can include both internal conditions and external surroundings. What matters is the interaction between the decision-making agent and the parts of the environment that influence its decisions.

The agent does not have to be a whole person or a robot. It can be a smaller decision-making system within a larger organism or machine. Similarly, the environment can include both internal conditions and external surroundings. What matters is the interaction between the decision-making agent and the parts of the environment that influence its decisions. Finally, reinforcement learning allows an agent to improve through experience. A chess player can improve their evaluation of positions, a young animal can become better at moving, and a person can become more efficient at completing everyday tasks. Previous knowledge or built-in abilities can influence learning, but interaction with the environment remains important for adapting behavior to the specific task.

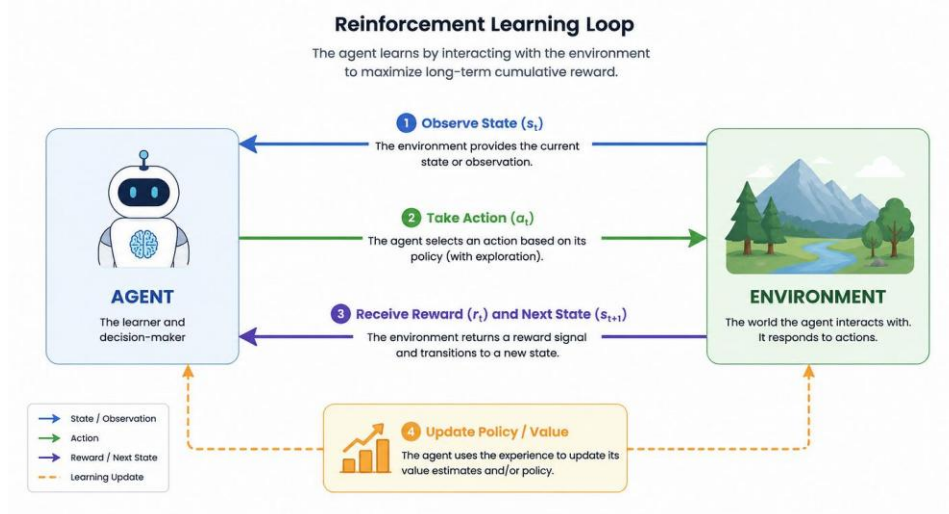


Figure 1. Reinforcement learning process

2. Elements of Reinforcement Learning

A reinforcement learning system consists of several fundamental elements that work together to enable an agent to learn through interaction with its environment. The two basic components are the **agent**, which makes decisions and selects actions, and the **environment**, which responds to those actions and provides new situations and rewards. In addition to these components, reinforcement learning is commonly described in terms of four main elements: the **policy**, **reward signal**, **value function**, and, optionally, an environment model. The policy determines how the agent selects actions in different states. The reward signal provides feedback about the desirability of the outcomes produced by the agent's actions. The value function estimates the expected long-term rewards associated with states or state-action pairs. When a model is available, it represents the environment's expected behavior, including how states may change and what rewards may result from different actions. Together, these elements provide the foundation for learning, decision-making, and goal-directed behavior in reinforcement learning.

3. Decision Making

Decision-making is the process of selecting an action from a set of possible options based on the current situation and the expected consequences of each. In reinforcement learning, decision-making occurs repeatedly because the action selected at a given time can influence the situations the agent encounters in the future. A decision-making problem, therefore, involves three important components:

1. **The current state** describes the situation in which the agent currently finds itself.
2. **The available actions** describe what the agent can do.
3. **The consequences of actions** determine the next state and the reward received by the agent.

The agent must therefore balance immediate benefits with possible future benefits.

4. Agent and Environment

The two fundamental components of reinforcement learning are the **agent** and the **environment**. The **agent** is the learner or decision maker who chooses actions. The **environment** is everything outside the agent that responds to the agent's actions. The basic interaction can be represented as:

State $\rightarrow$ Action $\rightarrow$ Reward $\rightarrow$ Next State

At time t , the agent observes a state S_t , selects an action A_t , and receives a reward R_{t+1} . The environment then transitions to a new state, S_{t+1} . Formally:

$$S_t \rightarrow A_t \rightarrow R_{t+1}, S_{t+1}$$

This interaction continues over multiple time steps.

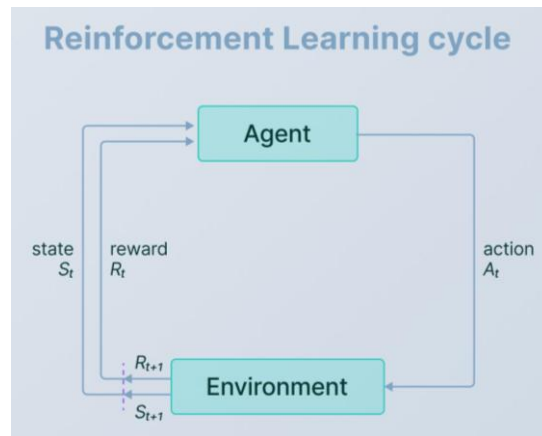


Figure 2. Reinforcement learning cycle

5. State

A **state** represents the information available to the agent about the current situation. The state is usually denoted by: S_t

For example, in a chess-playing system, the state can contain the positions of all pieces on the chessboard. In a self-driving vehicle, the state could contain information about the vehicle's position, velocity, nearby vehicles, traffic lights, and road conditions. A good state representation should contain enough information to make appropriate decisions about the future.

6. Action

An **action** is a decision made by the agent. The action is usually represented by " A_t ". The set of possible actions is called the action space. For example, a robot may have the actions:

$$A = \{\text{forward, backward, left, right}\}$$

Actions can be **discrete** or **continuous**. In a discrete action space, the agent selects one action from a finite set. In a continuous action space, the action can take a value from a continuous range, such as a vehicle's steering angle.

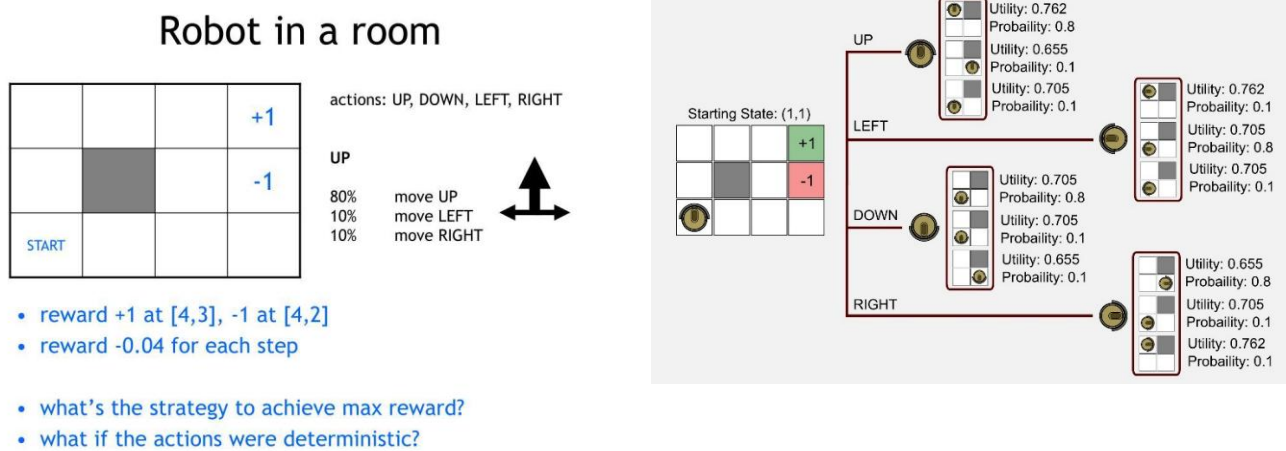


Figure 3. Example of a robot in a room

7. Reward

A **reward** is a numerical signal that indicates how desirable the outcome of an action is. The reward is represented by: R_{t+1}

A positive reward generally indicates a desirable outcome, while a negative reward indicates an undesirable one. For example, a game-playing agent might receive: +10 for winning and -10 for losing. The reward is extremely important because it defines what the agent is trying to achieve.

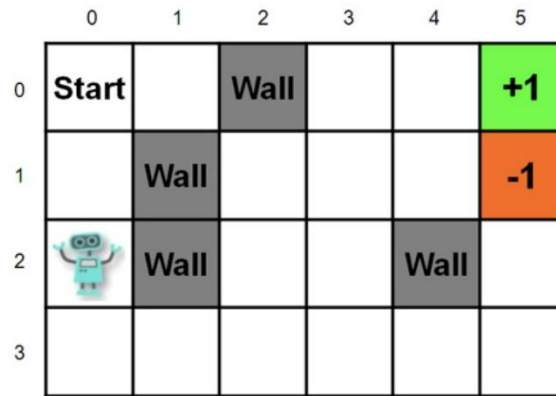


Figure 4. Example of a robot in a grid

8. Return

The **return** is the total amount of reward that the agent expects to receive from a particular time onward. A simple undiscounted return can be written as:

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + \dots$$

However, reinforcement learning commonly uses a **discount factor**. The discounted return is:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

where: $0 \leq \gamma \leq 1$ is the **discount factor**. A value of γ close to zero makes the agent focus more strongly on immediate rewards, while a value close to one makes the agent consider future rewards more strongly.

9. Policy

A **policy** defines how an agent chooses actions given its state. The policy is commonly represented by $\pi(a|s)$, which denotes the probability of selecting action a when the agent is in state s . A **deterministic policy** always selects the same action for a particular state: $a = \pi(s)$. A **stochastic policy** assigns probabilities to different actions:

$$\pi(a | s) = P(A_t = a | S_t = s)$$

The main objective of many reinforcement learning algorithms is to learn an **optimal policy**.

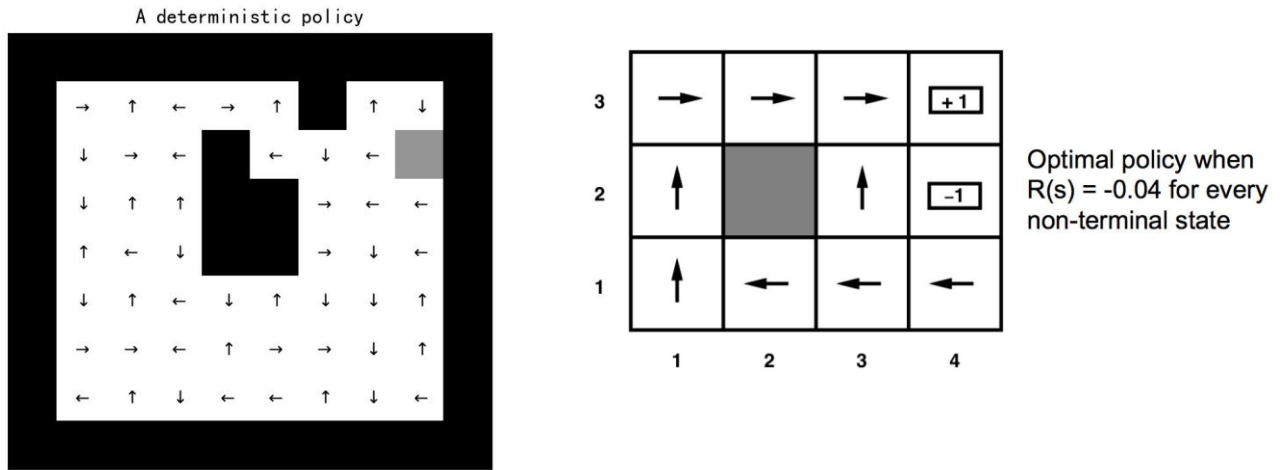


Figure 5. A deterministic and optimal policy

10. Value Function

A **value function** estimates how good a state is in terms of expected future rewards. The state-value function is defined as:

$$V^{\pi}(s) = E_{\pi} [G_t \mid S_t = s]$$

This means that $V^{\pi}(s)$ represents the expected return when the agent starts in state s and follows policy π . A state with a high value is expected to lead to large future rewards.

11. Action-Value Function

The **action-value function**, also called the **Q-function**, evaluates how good it is to take a particular action in a particular state.

It is defined as:

$$Q^{\pi}(s, a) = E_{\pi} [G_t \mid S_t = s, A_t = a]$$

The Q-function, therefore, answers the question:

"If I am in state s and take action a , how much future reward can I expect?"

The optimal action-value function is written as $Q^*(s, a)$ and represents the maximum expected return obtainable from state s after taking action a .

12. Markov Decision Process

Many reinforcement learning problems are modeled as Markov Decision Processes (MDPs). An MDP is commonly defined by the tuple: (S, A, P, R, γ) where:

- S represents the set of states.
- A represents the set of actions.
- P represents the transition probability.
- R represents the reward function.
- γ represents the discount factor.

The most important property of an MDP is the **Markov property**. The Markov property states that the future depends only on the current state and action, rather than on the entire history of previous states. Formally:

$$P(S_{t+1} | S_t, A_t, S_{t-1}, A_{t-1}, \dots) = P(S_{t+1} | S_t, A_t)$$

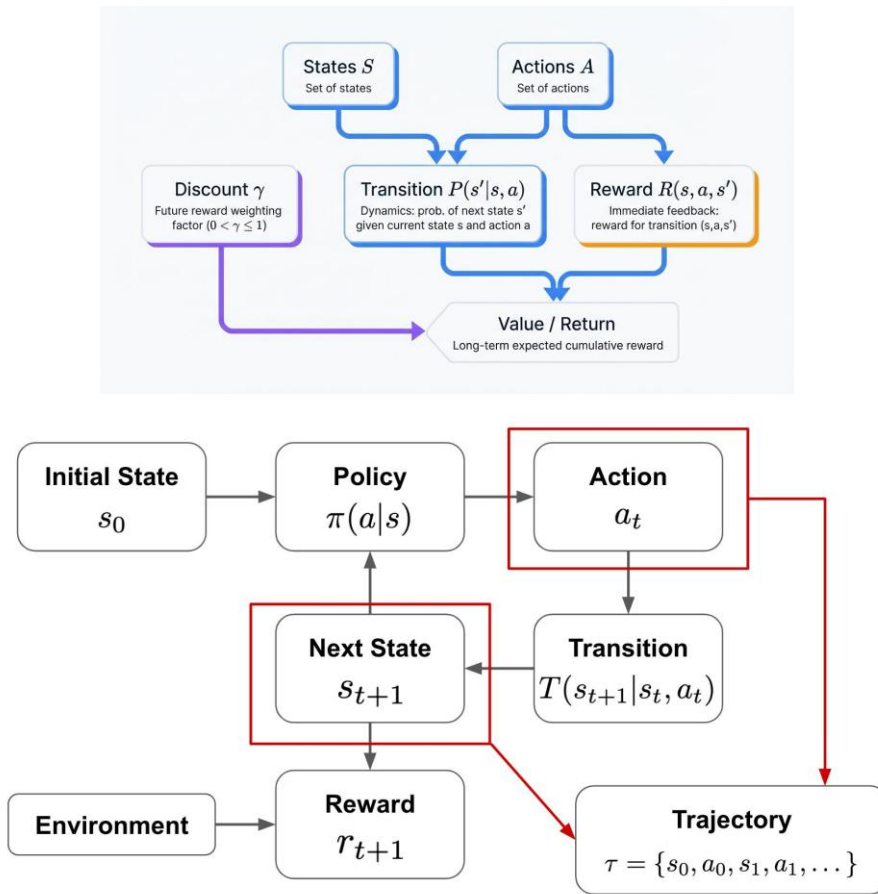


Figure 6. Markov Decision Process

13. Transition Probability

The **transition probability** describes the probability of moving from one state to another after an action is taken. It can be written as: $P(s' | s, a)$, where s is the current state, a is the selected action, and s' is the next state. If the environment is deterministic, a particular state-action pair may always produce the same next state. If the environment is stochastic, the same action can produce different next states with different probabilities.

14. Model-Based and Model-Free Reinforcement Learning

Reinforcement learning methods can broadly be divided into **model-based** and **model-free** approaches. A **model-based method** attempts to learn or use a model of the environment. The

model describes how actions affect future states and rewards. For example, the agent may learn: $P(s' | s, a)$ and: $R(s, a)$. The agent can then use this model to predict future outcomes before selecting an action. A **model-free method** does not explicitly learn the environment's transition dynamics. Instead, the agent learns directly from interaction and experience. Important model-free methods include: Monte Carlo methods, Temporal-Difference learning, SARSA, Q-learning, Deep Q-Networks, Policy-gradient methods, and Actor-Critic methods.

15. Exploration and Exploitation

One of the fundamental problems in reinforcement learning is the **exploration-exploitation dilemma**. **Exploration** means trying actions the agent knows little about to discover potentially better strategies. **Exploitation** means selecting actions that the agent currently believes will produce the highest reward. If the agent only exploits, it may never discover a better strategy. If the agent only explores, it may fail to take advantage of actions that are already known to be effective. A common method for balancing exploration and exploitation is the **epsilon-greedy strategy**. With probability ϵ , the agent chooses a random action, and with probability $1-\epsilon$, the agent chooses the action with the highest estimated Q-value.

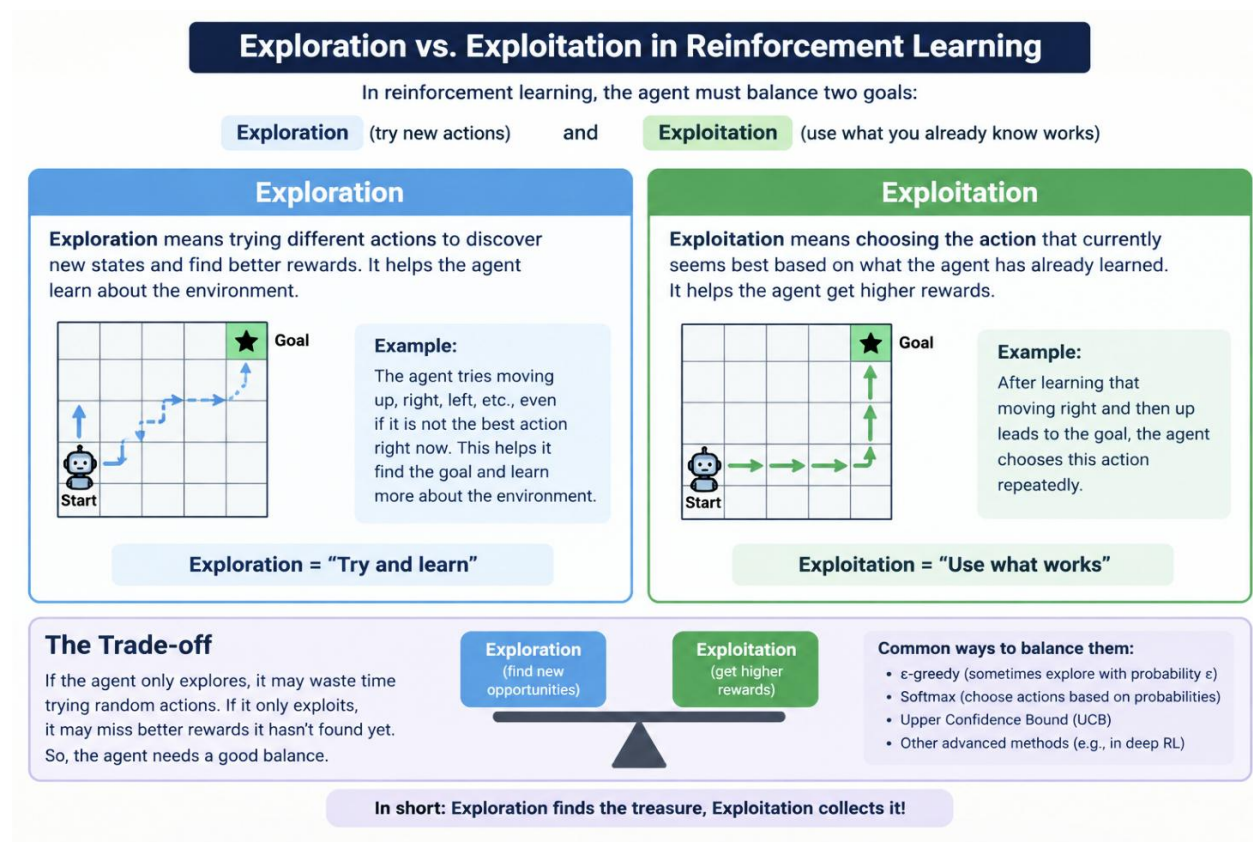


Figure 7. Exploration and Exploitation

16. Monte Carlo Methods

Monte Carlo reinforcement learning estimates values by waiting until an episode finishes and then calculating the actual return obtained during that episode. For example, if an episode produces rewards of 2, 3, and 5. Then the return from the first state, without discounting, is:

$$G_0 = 2 + 3 + 5 = 10$$

Monte Carlo methods, therefore, learn from **complete episodes**. Their main advantage is that they do not require a model of the environment. Their limitation is that they generally must wait until the end of an episode to update their estimates.

17. Temporal-Difference Learning

Temporal-Difference (TD) learning combines ideas from Monte Carlo learning and dynamic programming. Unlike Monte Carlo learning, TD learning does not need to wait until the episode ends. A basic TD update is:

$$V(S_t) \leftarrow V(S_t) + \alpha [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

where α is the learning rate. The quantity $\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$ is called the **TD error**. The TD error measures the difference between the current estimate and a new estimate based on the observed reward and next-state value.

18. Dynamic Programming

Dynamic Programming (DP) provides a group of methods for solving reinforcement learning problems when the environment model is known. Important dynamic programming methods include: **Policy Evaluation**, **Policy Improvement**, **Policy Iteration**, and **Value Iteration**. These methods use the Bellman equations to calculate or improve value functions and policies.

19. Bellman Equation

The **Bellman equation** expresses the value of a state in terms of the immediate reward and the value of future states. For a policy π , the Bellman expectation equation is:

$$V^\pi(s) = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) [r + \gamma V^\pi(s')]$$

The Bellman equation is fundamental to many reinforcement learning algorithms because it decomposes a long-term decision problem into smaller recursive problems.

20. Policy Evaluation

Policy evaluation calculates the value function associated with a given policy. The objective is to determine $V^\pi(s)$ for every state while keeping the policy π fixed. The agent, therefore, asks:

"How good is this particular policy?"

21. Policy Improvement

Policy improvement modifies a policy by selecting actions that have higher expected values. The basic idea is:

$$\pi'(s) = \arg \max_a Q^\pi(s, a)$$

This means the new policy selects the action with the highest estimated value. Policy evaluation and policy improvement together form the basis of **policy iteration**.

22. Policy Iteration

Policy iteration repeatedly performs two steps:

1. **Policy evaluation** estimates the value of the current policy.
2. **Policy improvement** produces a better policy using the estimated values.

The process continues until the policy no longer changes.

23. Value Iteration

Value iteration directly updates the value function toward the optimal value. The update is:

$$V_{k+1}(s) = \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma V_k(s')]$$

Value iteration does not completely evaluate each intermediate policy before improving it.

24. Q-Learning

Q-learning is an important **off-policy temporal-difference control algorithm**. It learns the optimal action-value function: $Q^*(s, a)$. The Q-learning update is:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

The important feature of Q-learning is that the update uses the **maximum Q-value of the next state**, regardless of the action actually selected by the behavior policy. Therefore, Q-learning is classified as an **off-policy** method.

25. 24. SARSA

SARSA is another temporal-difference control algorithm. The name SARSA comes from the sequence: $S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1}$. The SARSA update is:

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma Q(s', a') - Q(s, a)]$$

Unlike Q-learning, SARSA uses the **actual next action a'** selected by the policy. Therefore, SARSA is an **on-policy** method.

26. 25. On-Policy and Off-Policy Learning

An **on-policy algorithm** learns the same policy used to generate experience. SARSA is a classic example of an on-policy algorithm. An **off-policy algorithm** learns about a target policy while behavior may be generated using a different policy. Q-learning is a classic example of an off-policy algorithm. The distinction is important because it determines how the agent learns from exploratory actions.

27. 26. Deep Reinforcement Learning

Deep Reinforcement Learning (DRL) combines reinforcement learning with **deep neural networks**. Traditional reinforcement learning methods can struggle when the state space is extremely large. A neural network can approximate functions such as $Q(s, a)$ or $V(s)$, or directly represent $\pi(a|s)$. Deep reinforcement learning, therefore, enables RL to handle complex inputs such as images, audio, sensor measurements, and high-dimensional observations.

28. 27. Deep Q-Network

A **Deep Q-Network (DQN)** uses a neural network to approximate the Q-function. Instead of storing a Q-table, DQN learns: $Q(s, a; \theta)$, where θ represents the neural network parameters. Two important techniques used in the original DQN approach are: **Experience Replay** and **Target Network**.

29. 28. Experience Replay

Experience replay stores the agent's previous experiences in a replay buffer. An experience can be represented as: (s, a, r, s') . During training, the algorithm randomly samples experiences from this buffer. Experience replay helps reduce correlations between consecutive experiences, allowing the agent to reuse past experiences for learning.

30. 29. Target Network

A **target network** is a separate neural network used to calculate relatively stable target values during DQN training. The target network parameters are updated less frequently than the main network parameters. This technique helps improve training stability.

31. 30. Policy Gradient Methods

Policy gradient methods directly optimize the parameters of a policy. Instead of learning only a Q-function and then deriving a policy from it, the algorithm directly learns: $\pi_{\theta}(a|s)$, where θ represents the policy parameters. The objective is to maximize the expected return:

$$J(\theta) = E_{\pi_{\theta}}[G_t]$$

Policy-gradient methods are particularly useful when actions are continuous or when stochastic policies are desirable.

32. Multi-Armed Bandit

The **multi-armed bandit problem** is a simplified reinforcement learning problem in which the agent repeatedly chooses among several actions, called **arms**, and receives a reward.

Unlike a general MDP, the action normally does not cause a transition through a sequence of states.

The main challenge is again the **exploration-exploitation trade-off**.

Important bandit methods include:

- **Epsilon-greedy**
- **Upper Confidence Bound (UCB)**
- **Thompson Sampling**

33. Epsilon-Greedy

The **epsilon-greedy method** chooses a random action with probability ϵ and chooses the currently best-known action with probability $1-\epsilon$. This method is simple and widely used for exploration. A common strategy is to gradually decrease ϵ during training, a process called **epsilon decay**.

34. Upper Confidence Bound

The **Upper Confidence Bound (UCB)** method selects actions by considering both their estimated reward and the uncertainty of those estimates. The basic idea is to select the action with the highest optimistic estimate. This allows UCB to balance exploitation and exploration without choosing random actions as in epsilon-greedy.

35. Thompson Sampling

Thompson Sampling is a Bayesian approach to the multi-armed bandit problem. It maintains a probability distribution over the expected reward of each action. The algorithm samples from these distributions and selects an action based on the resulting estimates.

36. Episodic and Continuing Tasks

Reinforcement learning tasks can be divided into **episodic tasks** and **continuing tasks**. An **episodic task** has a clear terminal state, such as winning or losing a game. A **continuing task** does not have a natural terminal state, such as controlling a building's temperature. The definition of the return and the treatment of terminal states can differ between these two settings.

37. Terminal State

A **terminal state** is a state in which an episode ends. For example, in a maze, reaching the goal can be a terminal state. After reaching a terminal state, the agent typically does not take any further actions in that episode.

38. Learning Rate

The **learning rate**, usually represented by α , determines how strongly new information changes an existing estimate. A high learning rate makes the agent adapt quickly to new observations. A low learning rate makes learning more gradual. For example:

$$Q_{\text{new}} = Q_{\text{old}} + \alpha (\text{target} - Q_{\text{old}})$$

39. Discount Factor

The **discount factor**, represented by γ , determines the importance of future rewards. If $\gamma = 0$, the agent focuses only on immediate rewards. If γ is close to 1, the agent places substantial weight on future rewards. Therefore, γ controls the agent's effective planning horizon.

40. General Reinforcement Learning Framework

The complete reinforcement learning process can be summarized as:

$$S_t \rightarrow A_t \rightarrow R_{t+1}, S_{t+1} \rightarrow A_{t+1} \rightarrow \dots$$

The agent observes the state. The agent selects an action according to its policy. The environment produces a reward and a new state. The agent uses this experience to update its policy, value function, Q-function, or neural network. The process continues until the agent learns a policy that produces a high long-term return.

Exercises

Exercise 1 — Identify the RL components

A robot is placed in a 4×4 grid. It can move **up, down, left, or right**. It receives +10 when it reaches the goal, -10 when it enters a dangerous cell, and -1 for every normal movement.

Identify:

1. The agent.
2. The environment.
3. The state.
4. The action.
5. The action space.
6. The reward.
7. The goal of the agent.

Exercise 2 — RL or not RL?

For each problem, decide whether it is naturally formulated as a reinforcement-learning problem. Explain your answer in one or two complete sentences.

- A.** A model predicts whether an email is spam using thousands of labeled emails.
- B.** A robot learns which direction to move by receiving a reward when it gets closer to its destination.
- C.** A clustering algorithm groups customers according to their purchasing behavior without reward labels.
- D.** A chess-playing program learns by playing games and receiving a positive reward for winning.

Exercise 3 — State and action

Consider a mobile robot with:

- battery level = 40%;
- current position = (2,3);
- obstacle directly in front;
- charging station at (5,5).

Give one possible representation of the state.

Then give the possible actions if the robot can move: $A = \{\text{up, down, left, right, charge}\}$

Explain why the state should contain enough information for decision-making.

Exercise 4 — Reward design

Suppose you are designing an RL system for a robot that must reach a destination.

Design a reward function with:

- +100 for reaching the destination;
- -100 for collision;
- -1 for every movement.

Explain why giving -1 for every movement might encourage the robot to find shorter paths.

Exercise 5 — Calculate the return

An agent receives the following rewards: $R_1=2$, $R_2=4$, $R_3=-1$, $R_4=10$. Calculate the undiscounted return: $G_0 = R_1 + R_2 + R_3 + R_4$

Exercise 6 — Discounted return

Using the same rewards: 2, 4, -1, 10. calculate: $G_0 = R_1 + \gamma R_2 + \gamma^2 R_3 + \gamma^3 R_4$ for: $\gamma=0.9$. Show every calculation step.

Exercise 7 — Find the optimal action

An agent has the following Q-values:

State	Left	Right
S_1	4	7
S_2	10	6
S_3	-2	3

Write the optimal deterministic policy:

$$\pi^*(s) = \arg \max_a Q(s, a)$$

Exercise 8 — Markov Chain

Consider three states:

$$S = \{A, B, C\}$$

with transition matrix:

$$P = \begin{bmatrix} 0.5 & 0.5 & 0 \\ 0 & 0.2 & 0.8 \\ 0.3 & 0 & 0.7 \end{bmatrix}$$

Answer:

1. What is the probability of moving from A to B?
2. What is the probability of moving from B to C?
3. What is the probability of remaining in C?
4. Does each row sum to 1?
5. What does the matrix represent?

Exercise 9 — Exploration vs. exploitation

Suppose: $Q(s, \text{left}) = 10$ and: $Q(s, \text{right}) = 6$ with: $\epsilon = 0.2$

Under epsilon-greedy exploration:

1. What is the probability of exploitation?
2. What is the probability of exploration?

3. Which action is selected during exploitation?
4. Why can exploration still be useful even though its estimated Q-value is lower?

Exercise 10

Classify each method or situation as **model-based** or **model-free**.

1. An agent learns $P(s'|s, a)$ and uses it to predict future states.
2. An agent directly learns $Q(s, a)$ from experience without learning transition dynamics.
3. An agent uses a known transition matrix to calculate an optimal policy.
4. Q-learning.
5. An agent learns a model of a maze before planning a route.

Explain each answer.